Assignment 1

Samuel Ayers - 201330389

March 5, 2026

1 Multi-Armed Bandit

1.1 Problem Formulation

In this report, we will be considering the multi-armed or n-armed bandit learning problem. Here we are repeatedly faced with n options, choosing any one of these options will yield a reward taken from a stationary probability distribution. Our goal is to maximise our reward over the period we will be taking actions.

One way to conceptualise this problem is to imagine we are in a casino, there are n slot machines that we may play N times. Each slot machine calculates the reward it will give its user based on its own fixed and unique probability distribution. Naturally, we want to leave the casino with the greatest amount of money possible, and hence we must devise a strategy to maximise our profits.

As any casino-goer knows, unfortunately we do not know the expected reward from any of the slot machines, or the casino would likely lose money, similarly our agents will not know the expected reward of the actions in our experiment. Here, in our n-armed bandit problem, we can base our strategy on creating estimates for the expected reward from any given arm:

$$Q_t(a) \approx Q^*(a) = E(R_t|A_t = a)$$

Where $Q_t(a)$ is our estimate at time stamp t for the actual expected reward $Q^*(a) = E(R_t|A_t = a)$ of a given action, i.e. choosing arm a. We will model the n-armed bandit by first assigning each arm with a random $Q^*(a)$ value taken from a Gaussian distribution $\mathcal{N}(0, 1)$ with mean 0 and standard deviation 1, then the reward of choosing an arm will be selected randomly from a Gaussian Distribution $\mathcal{N}(Q^*(a), 1)$.

To create an experiment on this problem, we will also define multiple agents who will implement different strategies based on their estimations of $Q^*(a)$. The most basic of these will be the greedy method. This involves choosing the arm with the highest estimated value. Although this strategy works well when the optimal choice is known, it lacks any motivation to explore the different options available to it. This creates the possibility of only choosing a good action when an even better one exists, leading to a sub optimal result. Another strategy we will employ is known as ϵ-greedy, here we will still choose the action with the greatest estimated value but with probability ϵ we will choose a random action instead. This strategy allows the agent to estimate the expected reward more effectively for each arm. In our experiment we will test the greedy ($\epsilon = 0$) and ϵ-greedy strategy with different values for ϵ and compare the results.

1.2 Algorithm description

To solve our n-armed bandit problem, we will implement the strategies mentioned above. The objective being to maximise the cumulative reward over a fixed number of steps whilst balancing:

- Exploitation: Selecting the action with the highest estimated value
- Exploration: Selecting a random action to gather information

Our algorithm maintains an estimate of the expected reward for each action and records how many times the actual optimal value was chosen, after every action is completed.

We consider an n-armed bandit where:

- Each arm $a \in \{1, ..., n\}$
- The true action values $Q^*(a)$ are drawn from a Gaussian distribution:
$$Q^*(a) \sim \mathcal{N}(0, 1)$$
- Rewards R_t are generated from:
$$R_t \sim \mathcal{N}(Q^*(a), 1)$$
- The optimal action is defined as:
$$a^* = \arg\max_a Q^*(a)$$

Our agent will maintain:

- $Q_t(a)$: estimated value of action a
- $N_t(a)$: number of times action a has been chosen

After selecting action a_t and receiving reward R_t, we update $Q_t(a)$ as follows:

$$Q_{t+1}(a_t) = Q_t(a_t) + \frac{(R_t - Q_t(a_t))}{N_t(a)}$$

We select an action with the policy π:

$$\pi(t) = a_t = \begin{cases} \text{random action } a & \text{with probability } \epsilon \\ \arg\max_a Q_t(a) & \text{with probability } 1 - \epsilon \end{cases}$$

Note: for the greedy strategy $\epsilon = 0$

The complete algorithm is as follows:

1. Initialise:
   - $Q_0(a) = 0 \ \forall \ a$
   - $N_0(a) = 0 \ \forall \ a$
2. For each step $t = 0, ..., T$
   - Select action a with policy π
   - Obtain reward $R_t \sim \mathcal{N}(Q^*(a), 1)$
   - Update:
$$N_{t+1}(a_{t+1}) = N_t(a_t) + 1$$
$$Q_{t+1}(a_t) = Q_t(a_t) + \frac{(R_t - Q_t(a_t))}{N_t(a)}$$
   - Record:
     - Reward at each step
     - If optimal action was selected

To evaluate the performance of each agent we will calculate the average reward at each time step t, and the percentage of optimal action selection.

1.3 Implementation and Experiment

Our algorithm was implemented using Python, with the libraries, NumPy, Matplotlib, and Seaborn, for numerical computation and random sampling, plotting results and improved plot styling respectively. The implementation is modular, consisting of 2 main classes, Bandit and Agent. The Bandit class models the problem, housing the distributions for each arm and computing the rewards for each action. The Agent class implements the ϵ-greedy policy and stores the estimates for the value of each action, number of times each action is selected and the total number of actions taken.

The function `run_simulation()` performs our experiment. Here we used the following parameters:

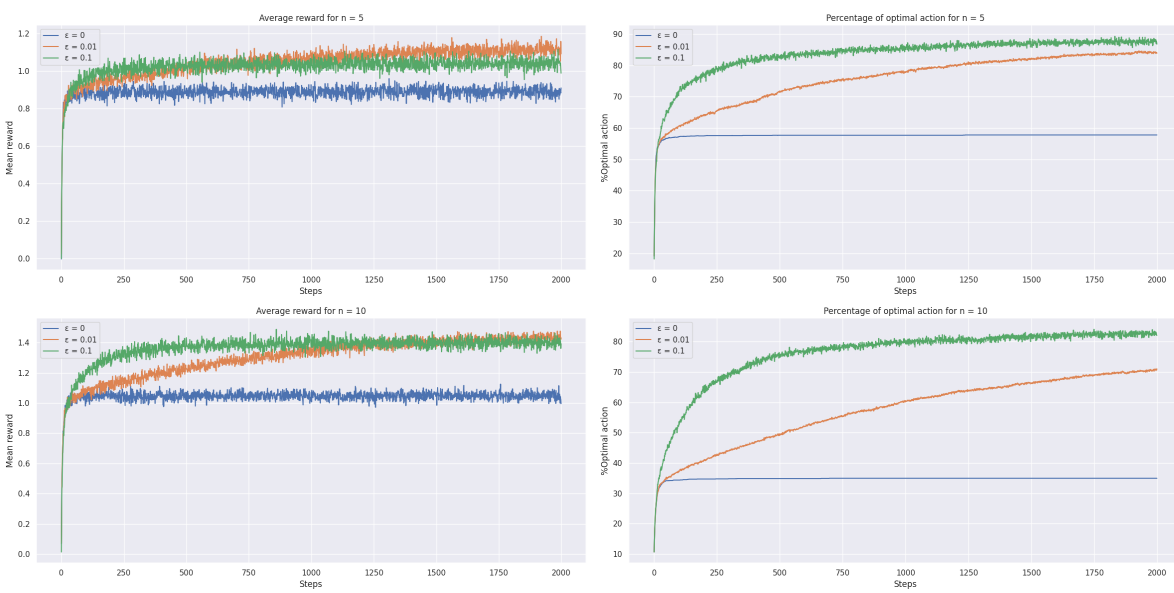
- Number of arms: $n \in \{5, 10, 20\}$
- Number of runs: 2000
- Number of steps: 2000
- ϵ values tested: $\epsilon \in \{0, 0.01, 0.1\}$

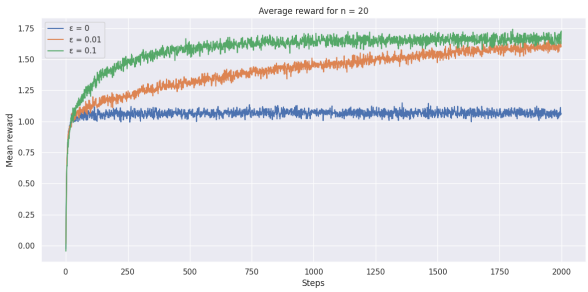
For each number of arms we tested, we performed 2000 runs, which involve the agent selecting and performing an action 2000 times. For each run the agents are reset and a new Bandit instance is created. It is necessary to run the experiment 2000 times as the multi-armed bandit problem is inherently stochastic, as a result of the this, performance on a single run can be extremely variable and may not reflect the true behaviour of the algorithm.

Finally the function `plot_results()` plots the evaluation metrics against steps for each n and ϵ value

1.4 Results and Discussion

After performing our experiment, we plotted the evaluation metrics: mean rewards and percentage of optimal action selection. This allows us to compare the effectiveness of the 3 different policies employed. Below you can see these plots.





From our plots, we observe, for all values of n and ϵ that initially all the agents perform similarly as the $Q_t(a)$ values are far from convergence so exploration is forced. We then see that the greedy method ($\epsilon = 0$) converges very early, at around 100 steps. This premature convergence however does not allow full exploration of all the arms leading to the agent choosing a suboptimal action to commit to. This results in a lower mean reward over time, with this strategy consistently being beaten by those with higher ϵ values. We can conclude that without exploration, the model cannot recover from early incorrect estimates of $Q^*(a)$. Now, comparing the two ϵ-greedy strategies, we see that the agent with $\epsilon = 0.1$ is the fastest to identify the optimal action, which follows from its higher rate of exploration. Once the $\epsilon = 0.01$ agent converges, we see for $n = 5, 10$ that it starts to outperform the $\epsilon = 0.1$ agent. This is due to both agents continuing to explore with their respective exploration rates, so although they have both identified the best action to take, the $\epsilon = 0.1$ model is only selecting that action 90% of the time and the $\epsilon = 0.01$ is selecting it 99% of the time. This explains the outperformance. In summary, the greedy method converges fastest, but has no guarantee to find the true optimal action. Both ϵ-greedy methods gradually increase optimal action selection over time. $\epsilon = 0.1$ converges faster initially but $\epsilon = 0.01$ achieves higher average reward in the long run.

Now we examine the difference between the different n values. As we increase n, the problem becomes more difficult, and more exploration is required to find the true optimal action. This causes convergence to take longer. With more arms, we see a larger gap between the performance of the greedy and ϵ-greedy agents, This is due to a greater chance of the greedy policy converging to an action other than the true optimal. We can conclude that, the importance of exploration increases with number of possible actions. We also see that the agent with the highest exploration rate retains the greatest average reward for the duration of our experiment for $n = 20$, given more steps however, I believe the $\epsilon = 0.01$ agent would eventually overtake it but due to the increase in convergence time, we do not observe this in our experiment.

The key observations from our experiment are that exploration is essential for discovering the optimal actions. Purely greedy strategies can work in simple problems, but especially in stochastic environments, they do not perform well. Higher exploration rate ϵ yields faster convergence but long term will achieve a lower average reward than a lower exploration rate. In addition to this, the larger the action space, the more exploration is required to find the optimal action. This problem perfectly illustrates the importance of balancing exploration and exploitation.

2 Exploitation vs Exploration

In reinforcement learning, exploitation refers to using knowledge of past events to choose the action that historically offers the greatest reward. In our experiment this is represented by choosing a_t such that:

$$a_t = \arg\max_a Q_t(a)$$

where $Q_t(a)$ is estimated reward of action a. The advantage of exploitation is immediate reward maximisation based on existing knowledge. If the estimates are accurate, exploitation yields high returns. However, exploitation carries a risk, if our estimates are inaccurate, the agent may commit to a suboptimal action, reducing the long term rewards i.e. we risk never discovering the true optimal action.

Conversely, exploration refers to selecting actions that are not currently considered optimal, in or-

der to gain information. In our experiment, we exhibit exploration by choosing a random action. Exploration is necessary because initially, agents have no knowledge of the environment and our early estimates are highly variable. Additionally, especially in stochastic environments like our n-armed bandit problem, the observed rewards contain noise. Exploration is what helps us to identify the true optimal action. By occasionally choosing actions other than the optimal, we reduce uncertainty and improve the accuracy of our value estimates.

2.1 The Exploitation-Exploration Dilemma

Any action selection must either be exploitative or explorative, so any policy must balance between obtaining short term rewards via exploration or long term knowledge via exploration. This is the exploitation-exploration dilemma. If the agent doesn't explore enough, it may converge to a suboptimal action, resulting in a decrease in long-term cumulative reward. If the agent explores too much, time is wasted selecting inferior actions and immediate rewards suffer. Operating with an extremely high exploration rate is similar to just choosing actions randomly. Therefore, an effective strategy must balance exploration and exploitation.

3 Action-Value Methods

Action-value methods are a class of reinforcement learning algorithms that estimate the expected reward of each action and then use these estimates to guide decision making. These estimates are updated solely based on observed rewards. This means the agent does not need to model the environment. If an action a has been selected k times, with corresponding rewards $r_1, r_2, ..., r_k$, we have $Q_t(a) = \frac{1}{k}\sum_{i=1}^{k} r_i$, by the law of large numbers we obtain:

$$\lim_{k\to\infty} Q_t(a) = Q^*(a)$$

To avoid storing all rewards, we can instead update our estimate with the following formula:

$$Q_{t+1}(a_t) = Q_t(a_t) + \alpha_t(R_t - Q_t(a_t))$$

For problems with stationary $Q^*(a)$ we can define $\alpha_t = \frac{1}{N_t(a)}$ i.e. sample-average step size.

3.1 Methods

Various action selection methods have been developed, they aim to address the exploration/exploitation dilemma. One example is the ϵ-greedy strategy, as we applied in our experiment. Another popular strategy is to employ an upper confidence bound (UCB). The aim of UCB is to, instead of indiscriminately choosing a random action with probability ϵ, select from the actions according to their potential for actually being optimal. We do this by accounting for how close the estimate is to being maximal and the uncertainty in that estimate. One example of this, as seen in [1] is to use the following formula to choose a_t:

$$a_t = \arg\max_a \left[Q_t(a) + \sqrt{\frac{ln(t)}{N_t(a)}} \right]$$

Here, the square root term represents the uncertainty of the estimate, this then gives an estimate for the possible true value of a. Each time a is chosen the uncertainty decreases and each time an action other than a is chosen, this term will grow. This method ensures that all actions will eventually be selected but as the number of steps increases, the wait will be longer.

Another method is Softmax (Boltzmann) action selection. Here we select actions probabilistically with probability:

$$P(a_t = a) = \frac{\exp(\frac{Q_t(a)}{\tau})}{\sum_b \exp(\frac{Q_t(a)}{\tau})}$$

Where τ is the temperature. When $\tau \to 0$ this method exhibits greedy behaviour, and when $\tau \to \infty$, uniform random behaviour. Softmax trades off exploration smoothly as we take more steps.

3.2 Strengths and Limitations

Action-value methods have many advantages. These methods are inherently simple, by estimating the expected return of each action and using these estimates to inform decision making, we avoid conceptually complex algorithms, making these methods accessible and interpretable. Since action-value methods don't require a model of the environment, they can be applied in situations where the environment is unknown or too complex to model explicitly. In addition to this, value estimates can be updated incrementally, removing the need to store past rewards, this is memory efficient, especially when dealing with long tasks.

Although there are many advantageous properties of action-value methods they also have limitations. The performance of action value methods is heavily reliant on handling exploration, a poor exploration strategy can lead to poor or suboptimal convergence or instability in learning. Environments with large action spaces also pose a challenge for action-value methods, increasing difficulty and computational cost. Action-value methods can also struggle when rewards are delayed or sparse, learning can become slow and unstable.

References

[1] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2 edition, 2018.